

Marc's Blog

About Me

My name is Marc Brooker. I've been writing code, reading code, and living vicariously through computers for as long as I can remember. I like to build things that work. I also dabble in machining, welding, cooking and skiing.

I'm currently an engineer at Amazon Web Services (AWS) in Seattle, where I work on databases, serverless, and serverless databases. Before that, I worked on EC2 and EBS. All opinions are my own.

Links

[My Publications and Videos](#)
[@marcbrooker on Mastodon](#) [@MarcJBrooker on Twitter](#)

It’s always TCP_NODELAY. Every damn time.

It's not the 1980s anymore, thankfully.

The first thing I check when debugging latency issues in distributed systems is whether [TCP_NODELAY](#) is enabled. And it’s not just me. Every distributed system builder I know has lost hours to latency issues quickly fixed by enabling this simple socket option, suggesting that the default behavior is wrong, and perhaps that the whole concept is outmoded.

First, let’s be clear about what we’re talking about. There’s no better source than John Nagle’s [RFC896](#) from 1984¹. First, the problem statement:

There is a special problem associated with small packets. When TCP is used for the transmission of single-character messages originating at a keyboard, the typical result is that 41 byte packets (one byte of data, 40 bytes of header) are transmitted for each byte of useful data. This 4000% overhead is annoying but tolerable on lightly loaded networks.

In short, Nagle was interested in better amortizing the cost of TCP headers, to get better throughput out of the network. Up to 40x better throughput! These tiny packets had two main causes: human-interactive applications like shells, where folks were typing a byte at a time, and poorly implemented programs that dribbled messages out to the kernel through many `write` calls. Nagle’s proposal for fixing this was simple and smart:

A simple and elegant solution has been discovered.

The solution is to inhibit the sending of new TCP segments when new outgoing data arrives from the user if any previously transmitted data on the connection remains unacknowledged.

When many people talk about Nagle’s algorithm, they talk about timers, but RFC896 doesn’t use any kind of timer other than the round-trip time on the network.

Nagle’s Algorithm and Delayed Acks

Nagle’s nice, clean, proposal interacted poorly with another TCP feature: delayed ACK. The idea behind delayed ACK is to delay sending the acknowledgement of a packet at least until there’s some data to send back (e.g. a telnet session echoing back the user’s typing), or until a timer expires. [RFC813](#) from 1982 is that first that seems to propose delaying ACKs:

The receiver of data will refrain from sending an acknowledgement under certain circumstances, in which case it must set a timer which will cause the acknowledgement to be sent later. However, the receiver should do this only where it is a reasonable guess that some other event will intervene and prevent the necessity of the timer interrupt.

which is then formalized further in [RFC1122](#) from 1989. The interaction between these two features causes a problem: Nagle’s algorithm is blocking sending more data until an ACK is received, but delayed ack is delaying that ack until a response is ready. Great for keeping packets full, not so great for latency-sensitive pipelined applications.

This is a point Nagle has made himself several times. For example in this [Hacker News comment](#):

That still irks me. The real problem is not tinygram prevention. It’s ACK delays, and that stupid fixed timer. They both went into TCP around the same time, but independently. I did tinygram prevention (the Nagle algorithm) and Berkeley did delayed ACKs, both in the early 1980s. The combination of the two is awful.

As systems builders this is should be a familiar situation: two reasonable features of the system that interact to create an undesirable behavior. This kind of interaction is one of the things that makes protocol design so hard.

Is Nagle blameless?

Unfortunately, it’s not just delayed ACK². Even without delayed ack and that *stupid fixed timer*, the behavior of Nagle’s algorithm probably isn’t what we want in distributed systems. A single in-datacenter RTT is typically around 500µs, then a couple of milliseconds between datacenters in the same region, and up to hundreds of milliseconds going around the globe. Given the vast amount of work a modern server can do in even a few hundred microseconds, delaying sending data for even one RTT isn’t clearly a win.

To make a clearer case, let’s turn back to the justification behind Nagle’s algorithm: amortizing the cost of headers and avoiding that 40x overhead on single-byte packets. But does anybody send single byte packets anymore? Most distributed databases and systems don’t. Partially that’s because they simply have more to say, partially its because of additional overhead of protocols like TLS, and partially its because of encoding and serialization overhead. But mostly, they have more to say.

The core concern of not sending tiny messages is still a very real one, but we’ve very effectively pushed that into the application layer. Sending a byte at a time wrapped in JSON isn’t going to be very efficient, no matter what Nagle’s algorithm does.

Is Nagle needed?

First, the uncontroversial take: if you’re building a latency-sensitive distributed system running on modern datacenter-class hardware, enable TCP_NODELAY (disable Nagle’s algorithm) without worries. You don’t need to feel bad. It’s not a sin. It’s OK. Just go ahead.

More controversially, I suspect that Nagle’s algorithm just isn’t needed on modern systems, given the traffic and application mix, and the capabilities of the hardware we have today. In other words, TCP_NODELAY should be the default. That’s going to make some “write every byte” code slower than it would otherwise be, but those applications should be fixed anyway if we care about efficiency.

Footnotes

1. I won’t got into it here, but RFC896 is also one of the earliest statements I can find of metastable behavior in computer networks. In it, Nagle says: “This condition is stable. Once the saturation point has been reached, if the algorithm for selecting packets to be dropped is fair, the network will continue to operate in a degraded condition.”
2. As this has gone around the internet, a number of folks have asked about TCP_QUICKACK. I don’t tend to reach for it for a few reasons, including lack of portability, and weird semantics (seriously, read [the man page](#)). The bigger problem is that TCP_QUICKACK doesn’t fix the fundamental problem of the kernel hanging on to data longer than my program wants it to. When I say `write()`, I mean `write()`.

« [Back to the blog index](#)

Similar Posts

- 21 Oct 2022 » [Give Your Tail a Nudge](#)
- 10 May 2014 » [The Essential Nancy Lynch](#)
- 20 May 2025 » [Good Performance for Bad Days](#)

Something Completely Different

- 11 Apr 2015 » [The Zero, One, Infinity Disease](#)

Marc Brooker
The opinions on this site are my own. They do not necessarily represent those of my employer.
marcbrooker@gmail.com

 [RSS](#)  [Atom](#)

This work is licensed under a [Creative Commons Attribution 4.0 International License](#).